Branch-free selections in production `Fp64`

`fp_backend::Fp64` and `prime_fast::Fp64` with `select_unpredictable`: results report, 2026-10-16

Study: `research/fp64_branchfree_20261016/`; protocol `PROTOCOL.md` frozen at `41a7a585c` before the first run. **Run report:** `reports/run-20261016T2319Z/run_report.pdf`. **Origin:** `hwcost` round 2 (`research/hwcost_round2_20261016/`, addendum 1). **Code:** `src/cryptanalysis/fp_backend.rs`, `src/cryptanalysis/prime_fast.rs`.

Scope. An engineering change to two field implementations, measured on `hwcost`'s rho kernel and on the production prime-field rho (`prime_fast::rho_parallel`). It changes no algorithm and no counted cost (every result is bit for bit the previous one), so it is not an index-calculus improvement, adds no gap-to-rho row and ranks no ECDLP methods (AGENTS.md success criterion, §5a).

1. Findings

1. **`fp_backend::Fp64`.** The rho kernel (`hwcost`, field $2^{63} - 25$) takes 0.4–0.44× of the baseline's time at every B ≥ 4096 in both pairs of runs (P1 PASS, worst 0.436 against 0.85). The candidate's kernel step compiles to `Fp64Cm`'s machine code (D2: 303 ⟶ 285 instructions, 32 ⟶ 20 conditional jumps, 4 ⟶ 12 `cmov`). It runs within 2.7% of `Fp64Cm` in the same runs (P3 PASS).
2. **`prime_fast::Fp64`.** The same kernel through `prime_fast`'s arithmetic (`pf64`, $2^{62} - 57$) takes 0.45–0.47× at every B ≥ 4096 (P1pf PASS, worst 0.466).
3. **The production rho.** `prime_fast::rho_parallel` on a 44-bit prime-order curve, one thread, 7 known-answer solves per run: the median goes from 98.9 to 68.2 ns per step in the first pair and from 98.6 to 71.1 in the second (ratios 0.689 and 0.721, drift 0.3%; P4 PASS against 0.95). That is 1.39–1.45× as many steps per second. Its worker loop `prime_fast::rho_worker` has 119 ⟶ 107 conditional jumps and 18 ⟶ 30 `cmov`.
4. **Guards and controls.** Dependent multiply and squaring chains move by at most 1.8% (P2a–d PASS against +10%). The unchanged `Fp64Cm` and `FpWide<1>` kernels stay within 2.5% and 3% (C1 PASS against ±10%). Memory bandwidth (20 arms) and clock (2 arms) show no detected effect. Inversion arms show no detected effect either (the binary extended Euclid is unchanged; only its final product is not).
5. **One cost, stated.** The `add.latency` arms time a dependent chain `a ← a − b` with a fixed `b`, whose wrap-around repeats in a pattern a predictor learns. On that chain the select costs latency where the branch was free:
   - `fp64`: 0.82/0.91 ⟶ 1.09/1.08 ns (ratios 1.33, 1.18), classed no detected effect only because the baseline drifted 12% between its runs;
   - `pf64`: ratios 1.26 and 1.27 (candidate slower).

   The same arm also moved for code that did not change (`fp64cm` 0.73, `w1` 0.87). This 1-ns chain is therefore sensitive to code placement between binaries, and part of the move may not be the change. Rho never runs such a chain: its subtractions take independent, effectively random operands.

The cost model agrees. The a priori op-count model (V2, no fitted parameter) missed the baseline `fp64` kernel by 55% and 54%, and `pf64` by 51% and 51% (round 1's `fp64` anomaly, which `pf64` shares). On the candidate it predicts `fp64` within 8% and 8%, and `pf64` within 6% and 7% (descriptive; `validation.json` per run). Counted operations now explain both production kernels.

Reading. Both production fields now do what round 2's `Fp64Cm` did, with the same results bit for bit, and the gain carries into the production rho: about 1.4× more steps per second for `prime_fast::rho_parallel` on this host. Every caller inherits it, including the F4 engine's row reductions, `two_sylow_search`, `Fp52`'s scalar operations and `prime_fast`'s index-calculus relation search, none of which was timed here.

Counted costs (GAE, field operations) do not change, so no index-calculus comparison in counted units moves. Wall-time comparisons against the `prime_fast` rho now face a faster rho.

Limits.

- Host L0: one shared 4-vCPU cloud VM with no PMU, so branch misses are inferred, not counted.
- Baseline-ISA builds only. A `-C target-cpu=native` build already turned some of these branches into `cmov` (round 2).
- `pfrho.44` is one curve size at one thread, with its set-up spread over a random number of steps per solve.
- `step_pf64` was never called, so it was linked in neither binary. It has been removed since, and `rho_worker` stands in for it in D2.

2. Claims

id	statement	value	threshold	verdict
P1	max(r1, r2) over 6 arms kernel.fp64.* (batch ≥ 4096)	0.4363	0.85	PASS
P1pf	max(r1, r2) over 6 arms kernel.pf64.* (batch ≥ 4096)	0.4655	0.85	PASS
P4	max(r1, r2) over 1 arms pfrho.44* (batch ≥ 0)	0.7212	0.95	PASS
P2a	max(r1, r2) over 1 arms fp64.mul.latency* (batch ≥ 0)	1.001	1.1	PASS
P2b	max(r1, r2) over 1 arms fp64.sqr.latency* (batch ≥ 0)	0.9968	1.1	PASS

P2c	max(r1, r2) over 1 arms pf64.mul.latency* (batch ≥ 0)	1.0184	1.1	PASS
P2d	max(r1, r2) over 1 arms pf64.sqr.latency* (batch ≥ 0)	0.9978	1.1	PASS
C1a	max |r_k − 1| over 6 arms kernel.fp64cm.* (batch ≥ 4096)	0.0247	0.1	PASS
C1b	max |r_k − 1| over 6 arms kernel.w1.* (batch ≥ 4096)	0.0299	0.1	PASS
P3	worst fp64/fp64cm kernel ratio at B ≥ 4096 over 2 runs (12 cells)	1.0269	1.1	PASS

Arm classes, candidate against baseline (an effect is called only when both ratios clear max(2·drift, 2%) on the same side):

arm group	candidate faster	candidate slower	no detected effect
clock	0	0	2
fp64	0	0	8
fp64cm	2	0	6
kernel	21	0	19
mem	0	0	20
pf64	0	1	7
pfrho	1	0	0
scaling	6	0	6
w1	1	0	7

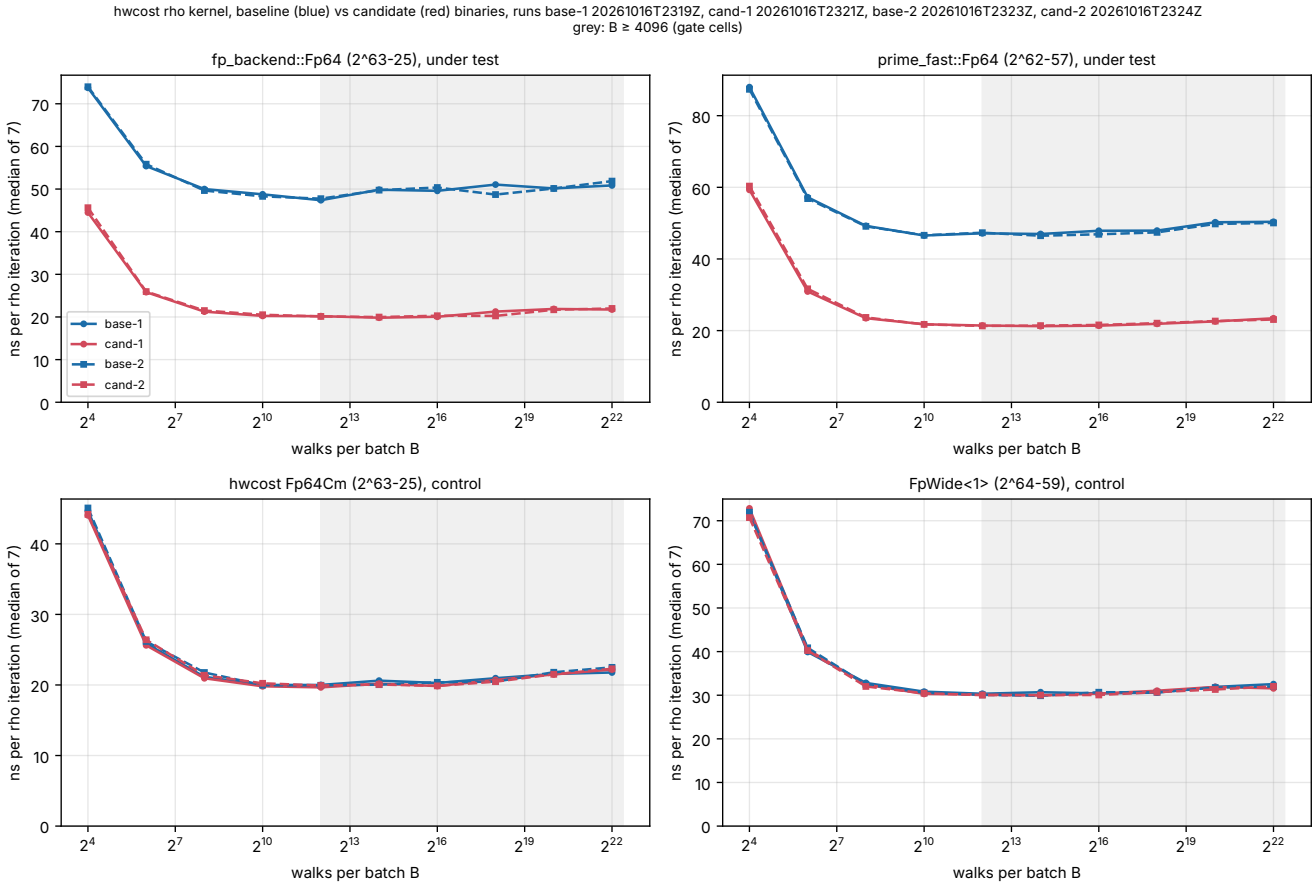


Figure 1: Rho kernel time per field, baseline (blue) and candidate (red) binaries, runs A1 B1 A2 B2.

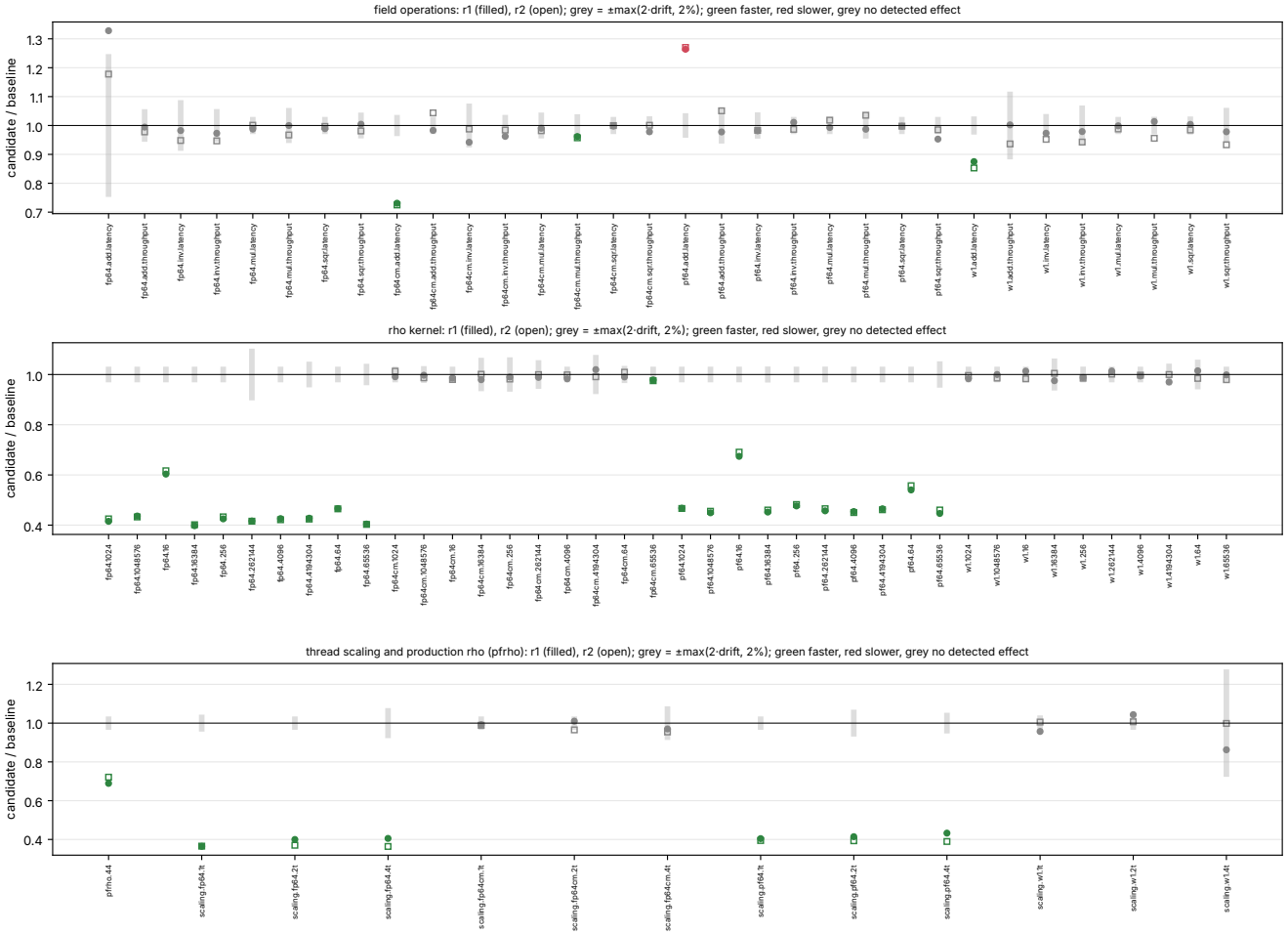


Figure 2: Candidate / baseline ratio for every arm.

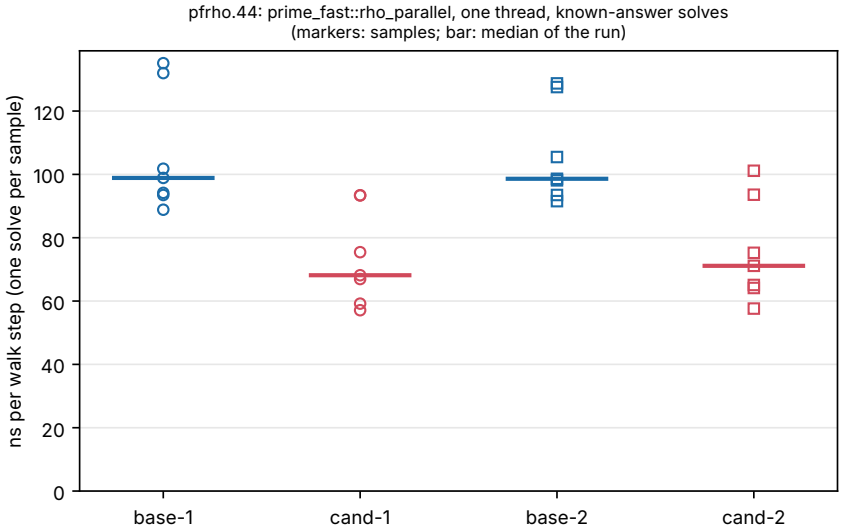


Figure 3: The production rho `prime_fast::rho_parallel` (44-bit curve, one thread): every sample.

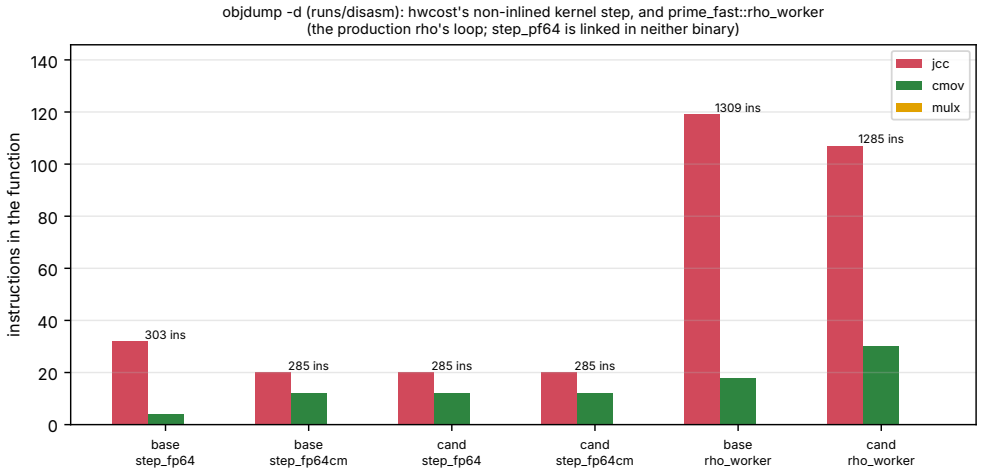


Figure 4: Machine code of `hwcost`'s non-inlined kernel step and of `prime_fast::rho_worker`, the production rho's loop.